```
function
ps_noise_filter(Nifgs,Nps_atmo,Nps_defo,Npsc_selections,Btemp,Bdop,defo_model_fl
ag,final_model,final_althyp_index,std_param,sig2_est,ts_noise_filter,ts_noise_fi
lter_length)

% ps_densification
%
% Input:    - Nifgs                   number of interferograms
%           - Nps_atmo                number of ps after
%                                     atmosphere modeling
%           - Nps_defo                number of ps after
%                                     deformation modeling
%           - Npsc_selections         number of psc selections
%           - Btemp                   temporal baselines [year]
%           - Bdop                    Doppler baselines [Hz]
%           - defo_model_flag         deformation model flag
%           - final_model             model used for unwrapped data
%           - final_althyp_index      alternative hypothesis of model
%           - std_param               standard deviations for pseudo-
%                                     observations
%           - sig2_est                estimated variance components
%           - ts_noise_filter         noise filter: block, triangle,
%                                     gaussian or prediction
%           - ts_noise_filter_length  noise filter length [year]
%
% ------------------------------------------------------------------------
% File.............: ps_noise_filter.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Astrid Humme
%                   Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%


global project_id m2ph weighting fig
global Npar_max Nlines Npixels max_mem_buffer

switch defo_model_flag
  case 'yes'
    results_id = [{'atmo'};{'defo'}];
    Nps = [Nps_atmo Nps_defo];
    Nresults = 2;
  case 'no'
    results_id = {'atmo'};
    Nps = Nps_atmo;
    Nresults = 1;
end

% ------------------------------------------------------------------------
% Construct stochastic model
% ------------------------------------------------------------------------

if isempty(sig2_est)
  if model(1) == 1 % master atmosphere stochastic
```

```
    sig2 = [(pi*15/180)^2; repmat((pi*20/180)^2,Nifgs,1)];
  else % master atmosphere estimated
    sig2 = [0; repmat((pi*30/180)^2,Nifgs,1)];
  end
else
  sig2 = sig2_est;
end

Qy = repmat(2*sig2(1),Nifgs,Nifgs);
for v = 1:Nifgs
  Qy(v,v) = Qy(v,v)+2*sig2(v+1);
end
invQy = inv(Qy);


% --------------------------------------------------------------------------
% Select desired model (for final estimation only, not unwrapping!)
% --------------------------------------------------------------------------

[Npar,par_index,covar_index] = ps_model_definitions('Npar',final_model);
[ps_annotation,par_index,covar_index] =
ps_model_definitions('annotation',final_model);
redun = Nifgs-Npar; %redundancy

althyp_index = final_althyp_index;
h2ph = repmat(NaN,Nifgs,1); %dummy value
[B1Qy2,par_index,covar_index,defo_index] =
ps_model_definitions('model',final_model,Nifgs,h2ph,Btemp,Bdop,std_param);
B1 = B1Qy2(1:Nifgs,:);
B1(:,1) = []; %remove height

switch weighting
 case 'unw'
  N = B1'*B1;
  R = chol(N);
  rhs =  R\(R'\B1');
 case {'weight','vce'}
  N = B1'*invQy*B1;
  R = chol(N);
  rhs =  R\(R'\(B1'*invQy));
end

% --------------------------------------------------------------------------
% Construct filter
% --------------------------------------------------------------------------

time_span = max(Btemp)-min(Btemp);
filter_nl = zeros(1,ceil(time_span*1000)+1);

switch ts_noise_filter

 case 'block'

  filter_nl(1:round(0.5*ts_noise_filter_length*1000)+1) = 1;

 case 'triangle'

  filter_nl(1:round(0.5*ts_noise_filter_length*1000)+1) =
fliplr(0:1/round(0.5*ts_noise_filter_length*1000):1);
  % factor 0.5 because of mirrored filter

 case 'gaussian'

  filter_length_ts = ts_noise_filter_length / 3; % 3*std
```

```
  filter_nl = 1/(sqrt(2*pi)*0.5*filter_length_ts*1000) * exp (-0.5*((0:
(ceil(time_span*1000)+1))/(0.5*filter_length_ts*1000)).^2);

 case 'prediction'

  error(['Noise filtering by least-squares prediction is not implemented' ...
       ' yet']);

end

for z = 1:Npsc_selections

  for w = 1:Nresults

    % --------------------------------------------------------------------
    % Determine buffersize
    % --------------------------------------------------------------------

    Nps_buffer = floor(max_mem_buffer/(5*(2*Nifgs+2)*8));

    if (Nps_buffer>=Nps(z,w))
      Nps_buffer = Nps(z,w);
      Nbuffers = 1;
      Nps_rem_buffer = 0;
    else
      Nbuffers = floor(Nps(z,w)/Nps_buffer);
      Nps_rem_buffer = rem(Nps(z,w),Nps_buffer);
      if (Nps_rem_buffer > 0)
        Nbuffers = Nbuffers + 1;
      end
    end

    Nps_buffer_orig = Nps_buffer;


    % --------------------------------------------------------------------
    % Open files
    % --------------------------------------------------------------------

    ps_fid = fopen([project_id '_ps_results_' char(results_id(w)) '_sel' ...
                    num2str(z) '.raw'],'r');

    ps_filt_fid = fopen([project_id '_ps_results_filt_' char(results_id(w)) ...
                         '_sel' num2str(z) '_' ts_noise_filter '_length' ...
                num2str(ts_noise_filter_length) '.raw'],'w');

    Nps_buffer = Nps_buffer_orig;
    Nps_count = 0;

    for v = 1:Nbuffers

      if (v == Nbuffers)&&(Nps_rem_buffer~=0)
        Nps_buffer = Nps_rem_buffer;
      end

      % --------------------------------------------------------------------
      % Read data
      % --------------------------------------------------------------------

      ps_data = fread(ps_fid,[6+Npar_max+3*Nifgs Nps_buffer],'double')';

      ps_az = ps_data(:,2);
      ps_r = ps_data(:,3);
      ps_defo_tot = ps_data(:,Nifgs+Npar_max+7:2*Nifgs+Npar_max+6);
```

```
clear ps_data

% ------------------------------------------------------------------------
% Filter noise timeseries
% ------------------------------------------------------------------------

ps_defo_filt = repmat(NaN,Nps_buffer,Nifgs);
for k = 1:Nifgs
weights = filter_nl(floor(abs(Btemp'-Btemp(k))*1000)+1);
ps_defo_filt(:,k) = (ps_defo_tot*weights')/sum(weights);
end

%for plot_id = 1:size(ps_defo_tot,1)
%  close all;
%  plot(Btemp, ps_defo_filt(plot_id,:),'o');
%  hold on;
%  plot(Btemp, ps_defo_tot(plot_id,:),'ro');
%  pause;
%end

% ------------------------------------------------------------------------
% Compute new ensemble coherence and sig2hat value
% ------------------------------------------------------------------------

y = m2ph*ps_defo_filt';
xhat = rhs*y;
yhat = B1*xhat;
ehat = y-yhat;

switch weighting
 case 'vce'
Tb = repmat(NaN,Nps_buffer,1);
for k = 1:Nps_buffer
  Tb(k) = ehat(:,k)'*invQy*ehat(:,k); % overall model test
end
sig2hat_filt = Tb/redun;        % estimate variance factor
otherwise
sig2hat_filt = repmat(NaN,Nps_buffer,1);
end

ps_ens_coh_filt = abs((1/Nifgs)*sum(exp(i*ehat'),2));

% ------------------------------------------------------------------------
% Write to file
% ------------------------------------------------------------------------

fwrite(ps_filt_fid,[ps_az ps_r ps_ens_coh_filt sig2hat_filt
ps_defo_filt]','double');

Nps_count = Nps_count+Nps_buffer;

if v==1

ps_random = unique(ceil(rand(10,1)*Nps_buffer));

for k = 1:length(ps_random)
  fig = fig+1;
  figure(fig);hold on;
  set(gcf,'visible','off');
  h1=plot(Btemp,1000*ps_defo_tot(ps_random(k),:),'r','linewidth',2);
  plot(Btemp,1000*ps_defo_tot(ps_random(k),:),'ro','linewidth',2);
  h2=plot(Btemp,1000*ps_defo_filt(ps_random(k),:),'g','linewidth',2);
  plot(Btemp,1000*ps_defo_filt(ps_random(k),:),'go','linewidth',2);
  grid on;
```

```
        xlabel('Temporal baseline [year]');
        ylabel('Displacement [mm]');
        title(['Deformation time series, PS ' num2str(ps_random(k)) ...
             ' (randomly selected,' char(results_id(w)) ')']);
        legend([h1 h2],[{'Original'},{'Filtered'}],'Location',...
             'SouthOutside','Orientation','horizontal');

        print('-dpng',['plots/noise_filter/' project_id '_ts_sel' ...
                   num2str(z) '_' char(results_id(w)) '_' ...
                         ts_noise_filter '_length' ...
                         num2str(ts_noise_filter_length) '_psc' ...
                   num2str(ps_random(k)) '.png']);

      end
      close all
      end

    end
  end
  fclose('all');
end
```